

Junior Cloud Security Engineer

Internship Task Report

1.0 Task Overview

Task ID	CSE-AWS-L-006-04
Task Title	Least-Privilege Policy Engineering and Permission Boundary Implementation
Skill Domain	AWS Identity and Access Management (IAM) / Cloud Security Architecture
Difficulty	Advanced
Duration	12 Hours
CPE Credits	4 CPE Credits
Cloud Platform(s)	AWS Free Tier
AWS Region(s) Used	us-east-2
Primary Tools	AWS CLI, IAM Access Analyzer, CloudFormation, Terraform, iam-vulnerable, VS Code, ChatGPT or Claude for policy reasoning
Target Environment	iam-vulnerable privesc-paths modules (controlled environment)
MITRE ATT&CK IDs	T1078 (Valid Accounts), T1098 (Account Manipulation), T1069.003 (Permission Groups Discovery), T1087.004 (Cloud Account Discovery)
Guiding Frameworks	CIS AWS Foundations Benchmark v1.5, NIST SP 800-53, ISO 27001, OWASP Cloud Top 10
Regulatory Alignment	ISO 27001 A.9 Access Control, A.12 Operations Security; NICE PR-CDA-001; SFIA Level 3-4

1.1 Introduction

1.1 Introduction

This task focused on strengthening AWS Identity and Access Management (IAM) security by applying the principle of least privilege, engineering restrictive IAM policies, and implementing permissions boundaries. The lab addressed common cloud security risks caused by excessive permissions and misconfigured IAM users and roles, which attackers may abuse to perform privilege escalation. These skills are important for cloud security engineers because properly designed IAM controls help reduce the attack surface and prevent compromised identities from gaining unauthorized administrative access.

1.2 Objectives & Learning Outcomes

Phase A — Theory & Conceptual Foundation

Objective: Understand the principle of least privilege and how permission boundaries enforce it in AWS environments.

How I met the objective: I reviewed AWS IAM concepts including identity-based policies, resource-based policies, managed policies, and permissions boundaries. I then created a custom JSON policy that provided read-only S3 permissions using `s3:ListBucket`, `s3:ListAllMyBuckets`, and `s3:GetObject`, without providing write or delete permissions. The completed policy was saved in VS Code as evidence.

Phase B — Environment Setup

Objective: Deploy and examine the IAM Vulnerable environment to understand how intentionally misconfigured IAM roles can introduce privilege-escalation paths.

How I met the objective: I cloned the BishopFox iam-vulnerable repository and examined its available privilege-escalation modules, including the `privesc-paths` resources. The current repository was Terraform-based rather than the Docker-based implementation described in the task instructions, so I initialized the environment using Terraform and reviewed the proposed deployment. Terraform identified 265 resources for the vulnerable IAM environment. I documented the repository/module configuration, Terraform initialization and deployment plan, as well as the Docker Compose incompatibility as supporting evidence.

Phase C — Core Execution: Policy Engineering

Objective: Engineer a least-privilege IAM policy that meets the business requirements of a Data Analyst who requires read-only access to an S3 data lake.

How I met the objective: I created `data_analyst_policy.json` with only the S3 permissions necessary to list the data-lake bucket and retrieve objects. I used the AWS IAM policy simulator to validate the policy. The simulation confirmed that `s3:GetObject` was **allowed**, while unnecessary write and delete permissions were excluded from the policy. This demonstrated how least-privilege policy engineering can provide required business access while limiting unnecessary capabilities.

Phase D — Permission Boundary Implementation & Threat Modeling

Objective: Apply an AWS permissions boundary to restrict administrative capabilities and reduce potential IAM privilege-escalation paths.

How I met the objective: I created the `BoundaryRestrictAdmin` customer-managed policy using `boundary_restrict_admin.json` and configured it as a permissions boundary for the IAM Vulnerable `privesc1-CreateNewPolicyVersion`-role. The boundary limited the maximum permissions available to the role and was designed to prevent administrative IAM actions that could be used for privilege escalation. I then used policy simulation to evaluate escalation-related actions such as `iam:CreatePolicyVersion`, `iam:SetDefaultPolicyVersion`, `iam:AttachRolePolicy`, and `iam:PutRolePolicy`, documenting the blocked actions as evidence of the boundary's effectiveness.

Overall Learning Outcomes

- **Apply the principle of least privilege in AWS IAM.** — I created restrictive S3 policies that granted only the permissions required by the specified business role.
- **Identify risks associated with excessive IAM permissions.** — I examined IAM Vulnerable privilege-escalation paths and identified how overly permissive users, roles, and policies can increase cloud security risk.
- **Engineer and validate custom IAM policies.** — I created a Data Analyst policy and validated its behavior using the AWS IAM policy simulator.
- **Implement permissions boundaries as a preventive security control.** — I created and attached a permissions boundary to an IAM Vulnerable sandbox role to limit its maximum effective permissions.
- **Evaluate IAM privilege-escalation risk and mitigation.** — I tested potentially dangerous IAM actions and documented how least privilege and permissions boundaries can reduce the ability of compromised identities to escalate privileges.

1.2 Scope & Legal Authorisation

All activities performed during this task were conducted strictly within authorised, isolated, and controlled lab environments for educational and cloud security training purposes. The IAM policies, permission simulations, Terraform configurations, and privilege-escalation testing were performed only against resources that were specifically created or configured for this lab.

Authorised Environments Used

- **Personal AWS Sandbox Account** — Used as the authorised AWS environment for IAM policy creation, AWS CLI testing, policy simulation, Terraform configuration, and permissions-boundary implementation.
- **Kali Linux Lab Environment** — Used as the local security workstation for AWS CLI commands, Terraform, Docker/Docker Compose testing, JSON policy development, and collection of technical evidence.
- **BishopFox IAM Vulnerable Environment** — The iam-vulnerable repository was used as an intentionally vulnerable AWS IAM training environment. The privesc-paths modules and associated sandbox IAM roles were examined for privilege-escalation risks.
- **AWS IAM Policy Simulator** — Used within the authorised AWS sandbox account to evaluate custom policies and confirm whether required and restricted actions were allowed or denied without performing the simulated actions against production resources.

Scope Restrictions

No external AWS accounts, production environments, third-party cloud infrastructure, or real customer data were intentionally accessed or tested during this task. Testing was limited to the authorised lab resources and IAM identities used for the exercise.

Potentially dangerous IAM operations were evaluated through controlled policy simulation where appropriate rather than being executed against real production resources. The permissions boundary and IAM policies created during the task were applied only to designated sandbox or intentionally vulnerable lab identities.

AI tools were used only to support analysis, explanation, troubleshooting, policy development guidance, and documentation. AI-generated responses were not treated as technical evidence, and the required evidence was obtained from the actual Kali Linux, AWS CLI, Terraform, VS Code, and AWS IAM lab environment.

1.3 MITRE ATT&CK Cloud Threat Mapping

MITRE ATT&CK Technique ID	Technique Name	Relevance to This Task
T1078.004	Valid Accounts: Cloud Accounts	The lab used AWS IAM users and roles to demonstrate how valid cloud identities can become a security risk when they are assigned excessive permissions. Least-privilege policies and permission boundaries were implemented to reduce the impact of compromised accounts.
T1098.003	Account Manipulation: Additional Cloud Roles	IAM privilege escalation can involve modifying cloud permissions or gaining access to additional privileged roles. The iam-vulnerable environment demonstrated how IAM role and policy misconfigurations can create these escalation opportunities.
T1548	Abuse Elevation Control Mechanism	The privesc-paths environment represented scenarios in which excessive IAM permissions could potentially be abused to gain higher privileges. Permission boundaries were implemented to restrict administrative IAM actions and reduce this risk.
T1530	Data from Cloud Storage Object	Phase C involved an S3 data lake where a Data Analyst required read-only access. The custom policy restricted access to s3:GetObject and s3:ListBucket while preventing unnecessary modification or deletion capabilities.
T1484.002	Domain or Tenant Policy Modification	IAM policies determine what AWS identities can perform. The lab examined the security impact of policy permissions and used permission boundaries to prevent roles from obtaining excessive administrative capabilities through IAM policy changes.

2.0 Environment & Tool Setup

2.1 Cloud Architecture Overview

The lab environment consisted of an authorised AWS sandbox account integrated with a Kali Linux workstation. AWS IAM was the primary cloud service used to create and evaluate policies, users, roles, and permissions boundaries, while AWS CLI and Terraform were used locally to interact with and assess the IAM environment. The BishopFox IAM Vulnerable environment provided intentionally vulnerable IAM roles and privilege-escalation paths for controlled security testing.

Cloud Resource / Component	Type / Service	Role in Task	Region / Config Notes
AWS Sandbox Account	AWS Cloud Environment	Isolated environment used for IAM policy engineering, simulation, and permissions-boundary testing	Authorised non-production lab account
IAM Users (temp-user, test-user)	AWS IAM	Used for AWS CLI authentication and controlled IAM testing	Lab-only IAM identities
Data Analyst Policy	AWS IAM Custom Policy	Implemented least-privilege S3 access using s3:ListBucket and s3:GetObject	Defined in data_analyst_policy.json
BoundaryRestrictAdmin	AWS IAM Customer-Managed Policy	Established maximum permissions and restricted administrative/privilege-escalation actions	Created from boundary_restrict_admin.json
privesc1-CreateNewPolicyVersion-role	AWS IAM Role / IAM Vulnerable	Intentionally vulnerable sandbox role used to evaluate the effect of a permissions boundary on an escalation path	Part of IAM Vulnerable privesc-paths environment
IAM Policy Simulator	AWS IAM	Tested whether S3 and administrative actions were allowed or denied without executing the simulated actions	Used through AWS CLI
IAM Vulnerable	Terraform / AWS IAM Lab	Provided intentionally vulnerable IAM roles and privilege-escalation scenarios	BishopFox lab repository
Terraform	Infrastructure as Code	Initialized and evaluated IAM Vulnerable infrastructure and identified proposed AWS resources	Executed from Kali Linux
AWS CLI	Command-Line Tool	Interacted with AWS IAM, created policies, attached boundaries, and performed policy simulations	Configured with dedicated lab profiles
VS Code	Local Development Tool	Created and reviewed IAM JSON policy files	Installed on Kali Linux

Cloud Resource / Component	Type / Service	Role in Task	Region / Config Notes
Kali Linux	Local Lab Workstation	Primary environment used to execute AWS CLI, Terraform, and policy-engineering tasks	Local isolated training environment

2.2 AWS CLI & Tool Configuration

The lab was performed primarily from Kali Linux using AWS CLI, Terraform, VS Code, and the AWS IAM Policy Simulator. Separate AWS CLI profiles were used to manage authentication during testing, and all policy and permissions-boundary activities were performed within the authorised sandbox AWS account.

Tool / Service	Version	Installation Method	Purpose in This Task
AWS CLI	AWS CLI v2 / installed version	Installed and configured on Kali Linux; credentials configured using aws configure and named profiles	Used to authenticate to the AWS sandbox, query IAM resources, create policies, attach permission boundaries, and run IAM policy simulations
AWS IAM Policy Simulator	AWS Service/API	Accessed through AWS CLI using simulate-custom-policy and simulate-principal-policy	Validated least-privilege policies and tested whether potentially excessive or privilege-escalation actions were allowed or denied
Terraform	Installed version	Installed on Kali Linux and initialized using terraform init	Used to initialize and examine the IAM Vulnerable infrastructure and generate the proposed IAM resource deployment plan
IAM Vulnerable	Git repository	Cloned from the BishopFox GitHub repository using git clone	Provided intentionally vulnerable AWS IAM roles and privesc-paths scenarios for studying privilege-escalation risks
Visual Studio Code	Installed version	Installed locally on Kali Linux	Used to create, edit, and review JSON policy files including data_analyst_policy.json and boundary_restrict_admin.json
Git	Installed version	Available through Kali Linux package environment	Used to clone the IAM Vulnerable repository
Docker / Docker Compose	Installed version	Installed/configured on Kali Linux	Evaluated for the Phase B deployment specified in the task brief; the current IAM Vulnerable repository did not contain the required Compose configuration

Tool / Service	Version	Installation Method	Purpose in This Task
Kali Linux	Installed lab version	Local virtual/lab operating system	Served as the primary workstation for AWS CLI, Terraform, Git, VS Code, and evidence collection

Key Configuration Commands Used

The following commands were used to configure and verify the Kali Linux and AWS environment before performing the IAM security tasks. Sensitive AWS credentials are intentionally excluded from the report.

AWS CLI Profile Configuration

A dedicated AWS CLI profile was configured for the sandbox environment:

```
aws configure --profile iam-lab-admin
```

The configuration included the sandbox IAM user's access credentials, default AWS region, and JSON output format. Access keys and secret keys are not included in the report for security reasons.

List Configured AWS Profiles

```
aws configure list-profiles
```

Output:

```
default
```

```
iam-lab-admin
```

```
adminrole
```

This confirmed that the dedicated iam-lab-admin profile was available for the lab.

Verify AWS Identity

```
aws sts get-caller-identity --profile iam-lab-admin
```

Output:

```
{
  "UserId": "[REDACTED]",
  "Account": "[REDACTED]",
  "Arn": "arn:aws:iam::[ACCOUNT-ID]:user/test-user"
}
```

This confirmed that the iam-lab-admin profile successfully authenticated as the authorised sandbox IAM user test-user.

Verify Terraform Installation

```
terraform --version
```

This command confirmed that Terraform was installed and available from the Kali Linux terminal.

Initialize Terraform Environment

```
cd ~/aws-iam-lab/iam-vulnerable
```

```
terraform init
```

Successful output included:

Terraform has been successfully initialized!

This confirmed that Terraform successfully initialized the IAM Vulnerable project and its required provider configuration.

Verify IAM Vulnerable Repository

```
cd ~/aws-iam-lab/iam-vulnerable
```

```
ls
```

The repository contained Terraform configuration files, modules, documentation, and supporting resources required for the IAM Vulnerable environment.

Verify Privilege-Escalation Modules

```
ls modules/free-resources/privesc-paths
```

This confirmed that the IAM Vulnerable privesc-paths resources were available locally for the lab.

Verify Docker Compose Availability

```
docker-compose --version
```

Output confirmed that Docker Compose was installed. However, running the deployment command from the supplied IAM Vulnerable repository:

```
docker-compose up -d
```

returned:

```
no configuration file provided: not found
```

This established that the current IAM Vulnerable repository did not contain the Docker Compose configuration referenced by the original Phase B instructions. The available repository instead used Terraform for AWS IAM resource deployment.

2.3 AI Tool Usage Declaration

AI was used as a supporting learning and troubleshooting resource during this task. All AWS commands were executed manually by the intern within the authorised lab environment, and all screenshots and technical outputs included as evidence were obtained directly from the lab environment.

AI Tool	Used For	What Was NOT Done with AI
ChatGPT/ Claude AI	Explaining AWS IAM concepts such as least privilege, identity/resource policies, and permissions boundaries; explaining IAM policy syntax; troubleshooting AWS CLI, Terraform, Docker Compose, and permission errors; interpreting policy simulation results; assisting with MITRE ATT&CK mapping; and improving report wording and documentation.	AI did not execute AWS CLI or Terraform commands, access the AWS account, perform privilege escalation, generate AWS Console/terminal screenshots, or fabricate technical findings or evidence. All commands were manually executed and all technical evidence was captured from the authorised lab environment.

3.0 Methodology — Phase Execution

Phase A: Theory & Conceptual Foundation

Phase Objective	Understand the principle of least privilege and how permission boundaries enforce it in AWS environments.
Estimated Time	60 minutes
Evidence File(s)	phaseA_s3_readonly_policy.png

Actions Taken

The first phase focused on understanding the principle of least privilege and how AWS IAM policies can be designed to reduce unnecessary access. I reviewed the concepts of identity-based policies, resource-based policies, AWS managed policies, customer-managed policies, and permissions boundaries. I also studied how excessive IAM permissions can increase the impact of a compromised cloud identity and create opportunities for privilege escalation.

The difference between identity-based and resource-based policies was reviewed. Identity-based policies are attached to IAM identities such as users and roles and specify the actions those identities can perform. Resource-based policies are attached directly to supported AWS resources and specify which principals can access those resources.

A simple S3 read-only policy was then created in JSON format. The policy allowed S3 listing and object-reading operations without granting object upload, modification, or deletion permissions.

Example policy actions included:

```
"Action": [  
    "s3:ListBucket",  
    "s3:GetObject"  
]
```

The policy was created and reviewed in VS Code to confirm the JSON structure and demonstrate the application of least privilege.

ChatGPT was used to help explain the difference between identity-based and resource-based IAM policies and to clarify least-privilege concepts. The information was reviewed and summarized in my own words for the report.

Key Observations

The main observation from this phase was that IAM permissions should be based on the actual requirements of an identity rather than providing broad service access. Permissions such as `s3:*` can expose unnecessary capabilities when a user only requires access to view data.

I also learned that an IAM permissions boundary does not directly grant access. Instead, it defines the maximum permissions an IAM user or role can receive. This distinction became important during the later privilege-escalation testing.

Evidence Produced

- **phaseA_s3_readonly_policy.png** — Screenshot showing the S3 read-only IAM JSON policy created and reviewed in VS Code.

Phase B: Cloud Environment Setup & Target Preparation

Phase Objective	Deploy the iam-vulnerable environment to simulate misconfigured IAM roles.
Estimated Time	90 minutes
Evidence File(s)	phaseB_terraform_init.png, phaseB_privesc_paths_modules.png, phaseB_terraform_plan.png, phaseB_docker_compose_error.png

Actions Taken

The IAM Vulnerable repository was cloned into the Kali Linux lab environment to provide an intentionally vulnerable AWS IAM environment.

```
git clone https://github.com/BishopFox/iam-vulnerable.git
```

```
cd iam-vulnerable
```

The supplied task instructions originally referenced Docker Compose and a local web interface. Docker Compose was verified; however, the current repository did not contain a Compose configuration file.

Attempting:

```
docker-compose up -d
```

returned:

no configuration file provided: not found

Review of the repository documentation established that the current IAM Vulnerable implementation uses Terraform to deploy vulnerable IAM resources into an authorised AWS sandbox account.

Terraform was therefore initialized:

```
terraform init
```

Successful output included:

Terraform has been successfully initialized!

The AWS identity was also verified before interacting with the environment:

```
aws sts get-caller-identity
```

The IAM Vulnerable modules were examined, including the available privesc-paths resources.

Terraform planning identified a large collection of intentionally vulnerable IAM resources:

Plan: 265 to add, 0 to change, 0 to destroy.

Some deployment operations were initially restricted by the permissions assigned to the AWS lab identity. These errors were documented rather than bypassed or concealed.

Key Observations

This phase demonstrated the importance of validating lab documentation against the actual software version being used. The task brief described a Docker-based environment, while the current IAM Vulnerable repository was Terraform-based.

The repository also demonstrated the size and complexity of the AWS IAM attack surface. A single intentionally vulnerable environment contained numerous users, roles, policies, and privilege-escalation paths.

The privesc-paths modules showed that privilege escalation is not necessarily caused by one obvious administrator permission. Combinations of IAM permissions can create paths through which a lower-privileged identity may obtain greater access.

Evidence Produced

- **phaseB_terraform_init.png** — Successful Terraform initialization of the IAM Vulnerable repository.
- **phaseB_privesc_paths_modules.png** — IAM Vulnerable privilege-escalation modules identified in the repository.
- **phaseB_terraform_plan.png** — Terraform plan showing the IAM resources proposed by the environment.
- **phaseB_docker_compose_error.png** — Evidence documenting that the current repository did not contain the Docker Compose configuration referenced by the task brief.

Phase C: Core Execution

Phase Objective	Engineer least-privilege policies based on business requirements.
Estimated Time	150 minutes
Evidence File(s)	phaseC_data_analyst_policy.png , phaseC_simulation_allowed.png , phaseC_simulation_denied.png

Actions Taken

The business scenario required a **Data Analyst** to have read-only access to data stored in an S3 data lake while preventing write and delete operations.

A custom least-privilege policy named:

data_analyst_policy.json

was created.

The relevant permissions were restricted to:

```
{
  "Effect": "Allow",
  "Action": [
    "s3:ListBucket"
  ],
  "Resource": "arn:aws:s3:::data-lake"
}
```

and:

```
{
  "Effect": "Allow",
  "Action": [
    "s3:GetObject"
  ],
  "Resource": "arn:aws:s3:::data-lake/*"
}
```

The policy was then evaluated using the AWS IAM policy simulator.

```
aws iam simulate-custom-policy \
  --profile iam-lab-admin \
  --policy-input-list "$(cat data_analyst_policy.json)" \
```

```
--action-names s3:GetObject \  
  
--resource-arns arn:aws:s3:::data-lake/sample.csv
```

The simulator returned:

```
"EvalActionName": "s3:GetObject"
```

```
"EvalDecision": "allowed"
```

This confirmed that the Data Analyst could retrieve an object from the specified data-lake resource.

Write and delete actions were also evaluated to confirm that permissions outside the business requirement were not granted.

During the initial simulation attempt, the lab identity was prevented from calling `iam:SimulateCustomPolicy`. IAM permissions were reviewed, and the lab configuration was corrected before repeating the simulation successfully.

Key Observations

This phase demonstrated that least privilege requires more than simply assigning a generic read-only managed policy. Permissions should be mapped directly to the business requirement and, where possible, restricted to specific resources.

The policy simulator provided a safe method of evaluating the expected effect of a policy without actually performing the S3 operation.

The successful `s3:GetObject` result demonstrated that required access remained available, while excluding write and delete actions reduced the impact that a compromised Data Analyst identity could have on cloud data.

Evidence Produced

- **phaseC_data_analyst_policy.png** — Custom least-privilege Data Analyst IAM policy.
- **phaseC_simulation_allowed.png** — AWS IAM policy simulator showing `s3:GetObject` evaluated as allowed.
- **phaseC_simulation_denied.png** — Simulation evidence showing unauthorized write/delete actions were not permitted.

Phase D: Independent Extension & Advanced Implementation

Phase Objective	Apply permission boundaries to restrict potential privilege escalation paths.
Estimated Time	180 minutes
Evidence File(s)	[e.g. <i>PhaseD_HardenedPolicy.json</i> , <i>PhaseD_RiskMatrix.xlsx</i> , <i>PhaseD_ThreatModel.png</i> , <i>PhaseD_AIAnalysis.txt</i>]

Actions Taken

Phase D focused on reducing IAM privilege-escalation risk by implementing a permissions boundary.

A boundary policy was created locally as:

```
boundary_restrict_admin.json
```

The boundary allowed only the workload permissions required for the exercise while excluding sensitive IAM administrative capabilities.

The policy was created in AWS as a customer-managed policy:

```
aws iam create-policy \  
  --profile iam-lab-admin \  
  --policy-name BoundaryRestrictAdmin \  
  --policy-document file://boundary_restrict_admin.json
```

AWS successfully returned the policy:

PolicyName: BoundaryRestrictAdmin

The IAM Vulnerable roles were then enumerated to identify a suitable intentionally vulnerable sandbox role.

```
aws iam list-roles \  
  --profile iam-lab-admin \  
  --query "Roles[?contains(RoleName, 'CreateNewPolicyVersion')].RoleName" \  
  --output text
```

The following role was identified:

privesc1-CreateNewPolicyVersion-role

The new policy was attached as the role's permissions boundary:

```
aws iam put-role-permissions-boundary \  
  --profile iam-lab-admin \  
  --role-name privesc1-CreateNewPolicyVersion-role \  
  --permissions-boundary arn:aws:iam::<ACCOUNT-ID>:policy/BoundaryRestrictAdmin
```

The boundary was then verified through AWS IAM.

Privilege-escalation-related permissions were evaluated against the restricted role, including:

```
iam:CreatePolicyVersion  
iam:SetDefaultPolicyVersion  
iam:AttachRolePolicy  
iam:PutRolePolicy
```

The results were documented to determine which administrative capabilities were prevented by the boundary.

Key Observations

This phase clarified the difference between an IAM identity policy and a permissions boundary. An identity policy can grant a permission, but the effective permission is still restricted by the maximum permissions defined by the boundary.

This behavior was also observed earlier when a permissions boundary prevented the lab identity from performing IAM operations even though an additional identity policy had been attached.

The IAM Vulnerable CreateNewPolicyVersion scenario also demonstrated why IAM policy-management permissions are security-sensitive. If an identity can modify policy versions or obtain broader policy-management capabilities, an attacker may be able to increase the privileges available to an identity.

Applying a restrictive permissions boundary therefore provides an additional preventive control for limiting the potential impact of excessive or incorrectly assigned identity permissions.

Evidence Produced

- **phaseD_boundary_policy.png** — JSON definition of the restrictive permissions boundary.
- **phaseD_boundary_policy_created.png** — AWS CLI confirmation that BoundaryRestrictAdmin was successfully created.
- **phaseD_boundary_attachment.png** — Confirmation that the boundary was attached to the IAM Vulnerable sandbox role.
- **phaseD_blocked_escalation.png** — Policy evaluation showing privilege-escalation-related administrative actions restricted by the permissions boundary.

Risk Matrix (Phase D Output)

Cloud Asset / Resource	Threat / Risk	MITRE Ref.	Likelihood (1–5)	Impact (1–5)	Score	Rating
IAM Vulnerable privesc1-CreateNewPolicyVersion-role	Privilege escalation through excessive IAM policy-version permissions	T1484.002 – Domain or Tenant Policy Modification	4	5	20	High
AWS IAM User / Role	Compromise or abuse of a valid cloud identity with excessive permissions	T1078.004 – Valid Accounts: Cloud Accounts	4	5	20	High
S3 Data Lake	Unauthorized access to cloud-stored data if an IAM identity receives excessive S3 permissions	T1530 – Data from Cloud Storage Object	3	4	12	Medium
IAM Role Policies	Unauthorized attachment or modification of IAM policies could increase a role's effective privileges	T1098.003 – Account Manipulation: Additional Cloud Roles	4	5	20	High
Permissions Boundary Configuration	Missing or incorrectly configured permissions boundaries could allow excessive administrative permissions to remain effective	T1484.002 – Domain or Tenant Policy Modification	3	5	15	High

Phase E: Hosted Lab Integration

Premium labs

Phase F: Documentation & Reporting

Phase Objective	<i>[State the objective of this phase as written in your task brief]</i>
Estimated Time	<i>[~60–90 minutes]</i>
Evidence File(s)	<i>[Filename(s) produced — e.g. PhaseA_Notes.pdf]</i>

Actions Taken

Phase F focused on consolidating the technical work completed throughout the AWS IAM least-privilege lab into a professional cloud security report. Evidence collected during Phases A–D was reviewed and organized to demonstrate the progression from IAM theory and environment setup through policy engineering, permissions-boundary implementation, and risk analysis.

The final report was structured to include an executive summary, objectives and learning outcomes, scope and legal authorization, MITRE ATT&CK mapping, environment and tool configuration, technical execution, policy design rationale, permissions-boundary configuration, risk matrix, findings, mitigations, recommendations, lessons learned, and supporting evidence.

Screenshots and CLI outputs were reviewed before inclusion to ensure that they corresponded to actual lab activities. Sensitive authentication information, including AWS secret access keys, was excluded from the report.

The policy-engineering section documented the rationale for the Data Analyst policy. The business requirement was translated into specific AWS permissions so that the analyst could list and retrieve S3 data while unnecessary write and delete capabilities were excluded.

The report also documented the implementation of the customer-managed:

BoundaryRestrictAdmin

permissions boundary and its application to the IAM Vulnerable sandbox role:

privesc1-CreateNewPolicyVersion-role

The risk matrix consolidated the main IAM security risks observed during the exercise and ranked them according to likelihood and impact. Relevant MITRE ATT&CK techniques were mapped to the identified risks to provide additional threat context.

The completed document was prepared for export using the required filename:

CSE_AWS_LeastPrivilege_Report.pdf

4.0 Findings

#	Finding Title	Type / Category	Cloud Service / Resource	MITRE Ref.	Phase	Severity	Evidence Ref.
1	Excessive IAM Permissions Create Privilege Escalation Paths	Privilege Escalation Path / IAM Policy Weakness	AWS IAM / IAM Vulnerable privesc-paths	T1484.002	B / D	High	phaseB_privesc_paths_modules.png, phaseD_blocked_escalation.png
2	IAM Policy Version Modification Presents Escalation Risk	IAM Policy Weakness / Privilege Escalation Path	privesc1-CreateNewPolicyVersion-role	T1484.002	D	High	phaseD_boundary_attachment.png, phaseD_blocked_escalation.png
3	Least-Privilege S3 Policy Successfully Restricts Data Analyst Access	Access Control / Least-Privilege Validation	Amazon S3 / Data Analyst Policy	T1530	C	Low / Informational	phaseC_data_analyst_policy.png, phaseC_simulation_allowed.png, phaseC_simulation_denied.png
4	Permissions Boundary Restricts Administrative IAM Capabilities	Access Control / Preventive Security Control	AWS IAM / BoundaryRestrictAdmin	T1484.002	D	Low / Informational	phaseD_boundary_policy_created.png, phaseD_boundary_attachment.png, phaseD_blocked_escalation.png
5	Valid Cloud Identities Require Effective Permission Governance	Access Control Weakness / IAM Governance	AWS IAM Users and Roles	T1078.004	C / D	Medium	IAM simulator outputs and permissions-boundary testing

4.2 Detailed Finding Analysis

Finding 1: Excessive IAM Permissions Create Privilege Escalation Paths

Field	Details
Finding Title	Excessive IAM Permissions Create Privilege Escalation Paths

Field	Details
Type / Category	IAM Policy Weakness / Privilege Escalation Path
Cloud Service / Resource	AWS IAM / IAM Vulnerable privesc-paths
Description	The IAM Vulnerable environment contained intentionally vulnerable IAM roles designed to demonstrate privilege-escalation paths. Review of the privesc-paths modules showed that sensitive IAM permissions can allow a lower-privileged identity to modify permissions or otherwise obtain increased access. The Terraform environment identified numerous IAM resources associated with these scenarios. This demonstrates the security risk created when IAM permissions are broader than required for an identity's business function.
How Identified	The IAM Vulnerable repository and its privesc-paths modules were reviewed from Kali Linux. Terraform initialization and planning were also performed to identify the IAM resources associated with the vulnerable environment.
MITRE ATT&CK Reference	T1484.002 – Domain or Tenant Policy Modification
Business / Security Impact	Excessive IAM permissions could allow a compromised identity to modify authorization controls and increase its effective privileges. Successful exploitation could affect the confidentiality and integrity of AWS resources and, depending on the permissions obtained, potentially provide broader control over the AWS account.
Severity / Impact	High
Evidence Reference	phaseB_privesc_paths_modules.png, phaseB_terraform_plan.png

Finding 2: IAM Policy Version Modification Presents Privilege-Escalation Risk

Field	Details
Finding Title	IAM Policy Version Modification Presents Privilege-Escalation Risk
Type / Category	IAM Policy Weakness / Privilege Escalation Path
Cloud Service / Resource	AWS IAM Role: privesc1-CreateNewPolicyVersion-role
Description	An intentionally vulnerable IAM role associated with creation of new policy versions was identified within the IAM Vulnerable environment. Policy-version management is security-sensitive because an

Field	Details
	identity capable of creating or changing policy versions may potentially alter the permissions represented by a managed IAM policy. The role was selected for Phase D to demonstrate how a permissions boundary can restrict the maximum permissions available to a potentially vulnerable identity.
How Identified	AWS IAM roles were enumerated using <code>aws iam list-roles</code> . A targeted query for <code>CreateNewPolicyVersion</code> identified <code>privesc1-CreateNewPolicyVersion-role</code> . The role was then selected for permissions-boundary testing.
MITRE ATT&CK Reference	T1484.002 – Domain or Tenant Policy Modification
Business / Security Impact	Unauthorized modification of IAM policies could allow an attacker to increase permissions available to a user or role. This could lead to privilege escalation, unauthorized access to cloud resources, modification of security controls, or further compromise of the AWS environment.
Severity / Impact	High
Evidence Reference	phaseB_privesc_paths_modules.png, phaseD_boundary_attachment.png

Finding 3: Data Analyst Access Successfully Restricted Using Least Privilege

Field	Details
Finding Title	Data Analyst Access Successfully Restricted Using Least Privilege
Type / Category	Access Control / Least-Privilege Validation
Cloud Service / Resource	Amazon S3 / data_analyst_policy.json
Description	The Data Analyst business requirement specified read-only access to an S3 data lake without write or delete capabilities. A custom policy was created containing <code>s3:ListBucket</code> for the data-lake bucket and <code>s3:GetObject</code> for objects within the bucket. Broad permissions such as <code>s3:*</code> were not granted. AWS IAM policy simulation confirmed that <code>s3:GetObject</code> against <code>arn:aws:s3:::data-lake/sample.csv</code> evaluated as allowed.
How Identified	The custom policy was validated using the AWS IAM <code>simulate-custom-policy</code> API through AWS CLI. The simulation evaluated the exact action and resource defined by the Data Analyst business requirement.
MITRE ATT&CK Reference	T1530 – Data from Cloud Storage Object

Field	Details
Business / Security Impact	Restricting the analyst to required read operations reduces the potential impact of account compromise. An attacker obtaining the Data Analyst identity would have fewer opportunities to modify or delete data through the permissions provided by this policy. This helps protect the integrity and availability of S3 data while maintaining legitimate business access.
Severity / Impact	Informational / Positive Security Control
Evidence Reference	phaseC_data_analyst_policy.png, phaseC_simulation_allowed.png

5.0 Risk Assessment & Cloud Threat Model

5.1 Cloud Security Control Gaps & Defensive Coverage

Finding / Gap	Observable / Trigger in Cloud Telemetry	Recommended Control	AWS Service / Tool for Implementation
Excessive IAM Permissions / Privilege Escalation Paths	IAM management events such as AttachRolePolicy, PutRolePolicy, CreatePolicyVersion, or SetDefaultPolicyVersion, particularly when performed by an unexpected identity	Apply least privilege; restrict sensitive IAM write operations; regularly review unused/excessive permissions; alert on high-risk IAM changes	AWS CloudTrail, IAM Access Analyzer, AWS Config, Amazon CloudWatch
IAM Policy Version Modification Risk	CloudTrail events showing CreatePolicyVersion, SetDefaultPolicyVersion, or other unexpected managed-policy modifications	Restrict policy-version management; use permissions boundaries and, where applicable, SCPs; alert on unauthorized policy changes	AWS CloudTrail, Amazon CloudWatch alarms, AWS Organizations SCPs, IAM permissions boundaries
Excessive S3 Data Access	Unexpected GetObject activity, unusual identities accessing S3 data, or attempts to perform PutObject/DeleteObject outside the expected role	Maintain resource-scoped least-privilege S3 policies; restrict write/delete permissions; periodically review access	AWS IAM, CloudTrail S3 data events, IAM Access Analyzer, S3 access controls
Missing/Weak Permissions Boundary	IAM role/user creation or modification without an expected permissions boundary; attempts to perform administrative IAM operations	Require restrictive permissions boundaries for delegated identities and prevent unauthorized removal or modification of boundaries	AWS IAM permissions boundaries, AWS Config, AWS Organizations SCPs, CloudTrail
Valid Cloud Identity with Excessive Permissions	Successful or denied API activity from an IAM identity outside its normal required operations; unexpected IAM administrative activity	Periodically review identity permissions, remove unused access, enforce least privilege, and monitor sensitive IAM API calls	IAM Access Analyzer, AWS CloudTrail, AWS Config, Amazon CloudWatch

5.2 Risk Exposure Over Time

Risk 1 — IAM Privilege Escalation Through Excessive Permissions

The IAM Vulnerable privesc-paths environment demonstrated that excessive IAM permissions can provide pathways for a lower-privileged identity to obtain greater access. If these

permissions remain unaddressed, a compromised IAM identity could potentially use sensitive policy-management capabilities to expand its privileges and maintain broader access to AWS resources. Over time, additional permissions or role relationships could increase the number of available escalation paths and increase the impact of an identity compromise. Applying least privilege and permissions boundaries reduces this exposure by limiting the maximum capabilities available to IAM identities.

Risk 2 — Unauthorized IAM Policy Version Modification

The `privesc1-CreateNewPolicyVersion`-role identified during the lab represents the risk associated with sensitive IAM policy-version management capabilities. If permissions such as `iam:CreatePolicyVersion` or `iam:SetDefaultPolicyVersion` are left available to identities that do not require them, an attacker who compromises such an identity could potentially modify authorization policies to obtain additional privileges. The resulting access could persist until the unauthorized policy changes are identified and reversed. The `BoundaryRestrictAdmin` permissions boundary implemented during Phase D was used to reduce this risk by limiting the role's maximum effective permissions.

Risk 3 — Excessive Access to S3 Data

The Data Analyst scenario demonstrated the importance of limiting S3 permissions to the specific operations required by a business role. If the analyst were instead granted broad permissions such as `s3:*`, compromise of that identity could expose not only data confidentiality through unauthorized object retrieval but also data integrity and availability through modification or deletion. As the amount of information stored in the data lake grows, the potential impact of excessive permissions would also increase. The custom `data_analyst_policy.json` reduced this exposure by limiting the role to required read operations such as `s3:ListBucket` and `s3:GetObject`.

6.0 Recommendations

6.1 Technical Remediation

#	Finding	Specific Remediation Action	Priority	Standard / Reference
1	Excessive IAM Permissions Create Privilege Escalation Paths	Review IAM users and roles and remove permissions not required for their defined function. Replace broad IAM permissions with explicitly permitted actions and resource ARNs. Use IAM Access Analyzer to identify unused access and apply permissions boundaries to identities where privilege escalation must be constrained.	Immediate	NIST CSF 2.0 PR.AA – Identity Management, Authentication, and Access Control; CIS AWS Foundations Benchmark – IAM controls
2	IAM Policy Version Modification Presents Privilege-Escalation Risk	Restrict sensitive actions including iam:CreatePolicyVersion and iam:SetDefaultPolicyVersion to specifically authorized administrative roles. Maintain a restrictive permissions boundary such as BoundaryRestrictAdmin on vulnerable/delegated roles and monitor these API calls through CloudTrail.	Immediate	NIST SP 800-53 AC-6 – Least Privilege; MITRE ATT&CK T1484.002 – Domain or Tenant Policy Modification
3	Data Analyst Access Successfully Restricted Using Least Privilege	Retain the resource-scoped data_analyst_policy.json approach. Permit only s3:ListBucket on the required bucket and s3:GetObject on its objects. Do not add s3:PutObject, s3:DeleteObject, or s3:* unless the business requirement changes. Re-test policy changes using IAM policy simulation before deployment.	Short-term / Ongoing	NIST SP 800-53 AC-6 – Least Privilege; MITRE ATT&CK T1530 – Data from Cloud Storage Object
4	Permissions Boundary Required to Restrict Administrative IAM Capabilities	Apply and maintain BoundaryRestrictAdmin on delegated or potentially sensitive IAM roles. Prevent unauthorized identities from removing or modifying the boundary. For organization-level environments, consider an SCP that restricts boundary removal and high-risk IAM administration.	Immediate	NIST SP 800-53 AC-6 – Least Privilege; NIST CSF 2.0 PR.AA; MITRE ATT&CK T1484.002

Technical Remediation Summary

The highest-priority remediation is to restrict IAM policy-management capabilities that could contribute to privilege escalation. The lab demonstrated that permissions such as policy-version creation and other IAM administrative operations require stronger controls because they can affect an identity's effective authorization.

The second key control is to maintain **permissions boundaries** for sensitive or delegated IAM roles. The BoundaryRestrictAdmin policy implemented during Phase D demonstrates how a maximum-permission ceiling can limit the impact of excessive identity-based permissions.

Finally, the Data Analyst policy should remain resource-scoped and read-only. The successful s3:GetObject simulation demonstrated that business requirements can be satisfied without granting broad S3 access.

6.2 Strategic & Operational Improvements

Based on the IAM privilege-escalation, least-privilege, and permissions-boundary findings from this task, the following broader improvements are recommended:

- **Establish an organization-wide IAM least-privilege review process.** Regularly review IAM users, roles, managed policies, inline policies, and trust relationships to identify unnecessary permissions. High-risk actions such as `iam:CreatePolicyVersion`, `iam:SetDefaultPolicyVersion`, `iam:AttachRolePolicy`, and `iam:PutRolePolicy` should be restricted to specifically authorized administrative identities.
- **Standardize permissions boundaries for delegated IAM roles.** Develop approved permissions-boundary templates similar to the `BoundaryRestrictAdmin` policy implemented during Phase D. Require these boundaries when creating developer, automation, or delegated-administration roles so that accidentally broad identity policies cannot exceed an approved maximum permission level.
- **Implement Service Control Policies for high-risk IAM operations.** In multi-account AWS environments, use AWS Organizations SCPs to restrict sensitive IAM administrative actions outside approved security or administration accounts. This [link](#) provides an additional organization-level control against the types of IAM policy manipulation and privilege-escalation paths examined in IAM Vulnerable.
- **Implement centralized monitoring for sensitive IAM API activity.** Record IAM management activity through CloudTrail and create monitoring/alerting for events such as `CreatePolicyVersion`, `SetDefaultPolicyVersion`, `AttachRolePolicy`, `PutRolePolicy`, and permissions-boundary changes. Unexpected use of these operations should be investigated because they may indicate unauthorized privilege modification.
- **Introduce policy validation into the IAM change process.** New or modified IAM policies should be reviewed and tested before deployment. IAM policy simulation and IAM Access Analyzer can be incorporated into the approval process to confirm required access while identifying unnecessary or overly broad permissions.
- The Data Analyst policy from Phase C demonstrates this approach by validating required `s3:GetObject` access without granting general S3 administration.

7.0 Reflection

Q1 How does implementing permission boundaries reduce the blast radius of compromised credentials?

Permission boundaries reduce the blast radius of compromised credentials by limiting the **maximum permissions** an IAM user or role can use. Even if credentials are stolen or an overly permissive policy is attached, actions outside the boundary remain blocked, reducing the risk of privilege escalation and broader AWS account compromise.

Q2 What trade-offs exist between operational flexibility and strict least privilege enforcement?

Strict least privilege improves security by reducing unnecessary access and limiting privilege escalation. However, overly restrictive permissions can slow operations, cause access-denied errors, and require frequent policy updates. A good balance is to grant only necessary permissions while regularly reviewing and adjusting access as business requirements change.

Q3 If a developer requests additional permissions citing productivity needs, how would you evaluate their request from a risk perspective?

I would evaluate whether the requested permissions are necessary for the developer's job responsibilities and identify the risks associated with granting them. I would prefer the **minimum required permissions**, limit access to specific resources, and use temporary access or permission boundaries where appropriate. The access should also be reviewed and removed when no longer needed.

Q4 How could automation tools like Access Analyzer be integrated into CI/CD pipelines for continuous compliance?

Access Analyzer can be integrated into CI/CD pipelines to automatically review IAM policies before deployment. If excessive or risky permissions are detected, the pipeline can flag or block the change, helping enforce least privilege and continuous compliance before misconfigurations reach production.

8.0 Deliverables Submission Checklist

Phase	Deliverable Description	Your Filename	Status
Phase A	Understand the principle of least privilege and how permission boundaries enforce it in AWS environments.	phaseA_s3_readonly_policy.png	Complete
Phase B	Deploy the iam-vulnerable environment to simulate misconfigured IAM roles.	phaseB_terraform_init.png, phaseB_privesc_paths_modules.png, phaseB_terraform_plan.png, phaseB_docker_compose_error.png	Complete
Phase C	Engineer least-privilege policies based on business requirements.	phaseC_data_analyst_policy.png, phaseC_simulation_allowed.png, phaseC_simulation_denied.png*	Complete*
Phase D	Apply permission boundaries to restrict potential privilege escalation paths.	phaseD_boundary_policy.png, phaseD_boundary_policy_created.png, phaseD_boundary_attachment.png, phaseD_blocked_escalation.png	Complete*
Phase E	Labs	N/A, Premium labs	No
Phase F	Final documentation containing findings, mitigations, risk assessment, recommendations and lessons learned	CSE_CSE-AWS-L-006-04_S.K_20260813.pdf	Complete

9.0 Additional Observations

Several troubleshooting situations during the task provided additional practical learning beyond the planned IAM activities.

- **Repository deployment mismatch:** The task instructions referenced Docker Compose and a local web interface for IAM Vulnerable. However, the current BishopFox IAM Vulnerable repository did not contain a Docker Compose configuration and instead used **Terraform** for deployment. After reviewing the repository documentation, Terraform was installed and successfully initialized, and the available privesc-paths modules were examined.
- **IAM permissions and permissions-boundary errors:** Several AWS CLI commands initially returned AccessDenied errors. For example, iam:SimulateCustomPolicy and IAM information commands were blocked because the active IAM user's permissions boundary did not allow the requested actions. Troubleshooting involved checking the active identity with aws sts get-caller-identity, reviewing configured AWS CLI profiles, and examining the user's identity policies and permissions boundary in the AWS Console.

- **AWS profile management:** Multiple AWS CLI profiles were present during the lab. Using `aws configure list-profiles` and `aws sts get-caller-identity --profile iam-lab-admin` helped confirm which identity was being used before performing security operations. This reinforced the importance of verifying credentials before making cloud configuration changes.
- **Local storage limitations:** The Kali Linux environment reached nearly 100% disk utilization during setup, which prevented creation of additional lab files and installation of required tools. Unneeded Burp Suite files, Docker/Juice Shop resources, caches, and unnecessary packages were removed, increasing available disk space and allowing the IAM lab to continue.
- **Policy simulation troubleshooting:** An initial `simulate-custom-policy` attempt produced an `InvalidInput` error because the JSON policy was not being passed correctly. Changing the command to pass the policy contents using `"$(cat data_analyst_policy.json)"` resolved the issue, and `s3:GetObject` subsequently evaluated as allowed.
- **Overall learning observation:** Troubleshooting these issues reinforced that cloud security engineering involves not only designing security controls but also understanding IAM authorization layers, CLI authentication, infrastructure-as-code tooling, environment dependencies, and interpreting error messages without weakening security controls unnecessarily.

Declaration & Sign-Off

I confirm that:

- All work in this report is my own except where AI usage is explicitly declared in Section 2.3.
- All technical activities were performed exclusively within the authorised cloud environments listed in Section 1.3.
- No external AWS accounts, production environments, third-party cloud resources, or real customer data were accessed during this task.
- All AWS credentials, access keys, session tokens, and account identifiers used during this task have been redacted in evidence files before submission.
- I have reviewed this report for accuracy, completeness, and professional presentation before submission.

Intern Full Name	Signature	Date of Submission
Sukhmandeep Kaur	Sukhmandeep Kaur	13 Aug 2026